

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases

Functions – Case Based Learning (BL 5)

Assessment Tool Weightage: 20%

QUESTIONS: 05

Total Marks:100

Annexure A

Industry Problem Solving

Code of Conduct:

I M. Vidhya(192412151) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1, A retail company wants to recommend products to customers based on past purchases. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity is measured, how recommendations are generated, and how the system adapts to new customer data.

Solution

A retail company can use the K-Nearest Neighbour (KNN) algorithm to recommend products based on customers' previous purchases. Each customer is represented using features such as product categories purchased, purchase frequency, amount spent, and ratings.

Step 1: Collect Customer Data

The system collects historical data such as:

- Products purchased
- Number of purchases
- Purchase frequency
- Amount spent
- Product categories
- Customer ratings

For example:

Customer	Electronics	Clothing	Books	Spending	Class
C1	8	2	1	50000	Electronics
C2	7	3	1	45000	Electronics
C3	1	8	2	25000	Clothing
C4	2	7	3	28000	Clothing

Step 2: Measure Similarity

Euclidean distance can be used to measure similarity between customers:

A smaller distance means greater similarity.

For example, if a new customer has purchasing behaviour similar to C1 and C2, those customers become its nearest neighbours.

$$d(X, Y) = \sqrt{\sum_{i=1}^n (X_i - Y_i)^2}$$

Step 3: Select K Neighbours

The system calculates distances from the new customer to all existing customers and selects the K closest customers.

For example, if: $K = 3$

the three most similar customers are selected.

Step 4: Generate Recommendations

The products purchased by the nearest neighbours but not yet purchased by the new customer are recommended.

For example:

- Similar Customer 1 → Laptop, Mouse
- Similar Customer 2 → Laptop, Keyboard
- Similar Customer 3 → Mouse, Headphones

Therefore, the system may recommend:

Laptop, Mouse, Keyboard and Headphones.

Step 5: Adapt to New Customer Data

Whenever a new customer makes a purchase, the customer's information is added to the dataset. KNN does not require complete retraining because it is an instance-based learning algorithm. Future recommendations can therefore use the newly added customer information.

Advantages

- Simple to implement
- No complex training phase
- Can provide personalized recommendations
- Easily adapts to new customer data

Conclusion

KNN provides personalized product recommendations by finding customers with similar purchasing behaviour. Euclidean distance measures similarity, and products purchased by nearby customers are recommended to the target customer. The system continuously improves as new customer data becomes available.

2. A manufacturing industry needs to predict machine output based on sensor readings that vary over time. Propose a solution using Locally Weighted Regression. Explain how local models improve prediction accuracy and how the system handles noisy or incomplete data.

Solution

A manufacturing company can use Locally Weighted Regression (LWR) to predict machine output from sensor readings such as temperature, pressure, vibration, speed, and operating time.

Unlike ordinary regression, LWR gives higher importance to data points close to the query point.

Step 1: Collect Sensor Data

Sensors continuously collect:

- Temperature
- Pressure
- Vibration
- Motor speed
- Operating time
- Machine output

Example:

Temperature Pressure Vibration Speed Output

50	20	2.1	1000	80
55	22	2.4	1100	85
60	25	2.8	1200	92
70	30	3.5	1400	105

Step 2: Select Query Point

Suppose the system needs to predict machine output for a particular sensor reading.

The query point is compared with historical observations.

Step 3: Calculate Weights

LWR assigns larger weights to nearby observations and smaller weights to distant observations.

A common Gaussian weighting function is:

$$w_i = \exp(-2\tau^2(x_i - x_q)^2)$$

where:

- x_i = training data point
- x_q = query point
- τ = bandwidth parameter
- w_i = weight

Step 4: Build Local Model

A regression model is fitted using the weighted data.

Nearby machine conditions receive high weights, so the model mainly learns the behaviour around the current operating condition.

How Local Models Improve Accuracy

Machine behaviour may change under different operating conditions. A single global regression model may not represent all conditions accurately.

LWR solves this problem by creating a local model for each prediction.

For example:

- Low temperature → local relationship
- Medium temperature → another local relationship
- High temperature → another local relationship

Therefore, LWR can capture changing machine behaviour more accurately.

Handling Noisy Data

Noise can be reduced because observations far away from the query point receive smaller weights.

The system can also:

- Remove obvious outliers
- Smooth sensor readings
- Use appropriate bandwidth values
- Give lower importance to unreliable observations

Handling Incomplete Data

Missing sensor values can be handled using:

- Mean/median imputation
- Interpolation for time-series data
- Previous valid sensor values
- Removing observations with excessive missing data

Advantages

- Captures local behaviour
- Suitable for changing machine conditions
- Reduces the influence of distant observations
- Useful for time-varying sensor data

Conclusion

Locally Weighted Regression predicts machine output by giving greater importance to sensor readings that are similar to the current operating condition. This local approach improves prediction accuracy when machine behaviour changes over time and can be combined with preprocessing techniques to handle noisy and incomplete sensor data.

3. A healthcare company aims to classify patients into risk categories using historical medical data. Suggest a model using KNN or RBF networks. Explain how the algorithm handles large datasets, feature selection, and accuracy improvement in real-world scenarios.

Solution

A healthcare company can use K-Nearest Neighbour (KNN) to classify patients into risk categories such as:

- Low Risk
- Medium Risk
- High Risk

Historical medical records are used as training examples.

Step 1: Select Features

Patient data may contain:

- Age
- Blood pressure
- Blood glucose
- Cholesterol
- BMI
- Heart rate
- Previous medical history

For example:**Patient Age BP Glucose BMI Risk**

P1	30	120	90	22	Low
P2	45	140	130	28	Medium
P3	60	160	180	32	High

Step 2: Feature Selection

Not every feature is equally useful. Irrelevant features can reduce classification accuracy.

Important features can be selected using:

- Correlation analysis
- Statistical tests
- Feature importance
- Domain knowledge

Step 3: Normalize Features

Healthcare features have different scales. For example, age may range from 0–100, while glucose may range from 50–300.

Therefore, normalization is required.

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Min-max normalization: $x' =$

Step 4: Calculate Distance

Euclidean distance can be calculated:

$$d(X, Y) = \sqrt{\sum_{i=1}^n (X_i - Y_i)^2}$$

The K nearest patients are selected.

Step 5: Classify Patient

Suppose:

If the five nearest patients contain:

- 3 High Risk
- 1 Medium Risk
- 1 Low Risk $K = 5$

then the new patient is classified as:

High Risk

because it has the majority vote.

Handling Large Datasets

Basic KNN can become slow for large datasets because it calculates distances to many training examples.

The system can improve performance using:

- KD-Trees
- Ball Trees
- Approximate nearest-neighbour search
- Dimensionality reduction
- Feature selection

Improving Accuracy

Accuracy can be improved by:

1. Selecting medically relevant features.
2. Normalizing the data.
3. Choosing an appropriate value of K.
4. Removing noisy/outlier records.
5. Using cross-validation to select K.
6. Handling missing values properly.

Real-World Application

When a new patient's medical information is entered, the system compares it with historical patients and identifies the most similar cases. The majority risk category of these patients becomes the predicted risk category.

Conclusion

KNN can classify patients into risk categories by comparing them with similar historical patients. Feature selection, normalization, appropriate K selection, and efficient nearest-neighbour search improve accuracy and performance, especially for real-world healthcare datasets.

4. An e-commerce platform wants to detect similar products for dynamic pricing strategies. Design a solution using Case-Based Learning. Discuss how similarity metrics are defined, how past cases are reused, and how the system scales with increasing product data.

Solution

An e-commerce company can use Case-Based Learning (CBL) to identify products that are similar to a target product. Previous product cases are stored in a case database and reused when a new product is analyzed.

Step 1: Store Previous Cases

Each product case can contain:

- Product category
- Brand
- Price
- Size
- Colour
- Features
- Customer ratings
- Previous sales
- Demand

For example:

Product Category Price Rating Storage Demand

P1	Mobile	₹20,000	4.5	128GB	High
P2	Mobile	₹22,000	4.4	128GB	High
P3	Laptop	₹50,000	4.2	512GB	Medium

Step 2: Define Similarity

A similarity score can be calculated using several product attributes.

For numerical attributes:

$$Similarity = 1 - \frac{|x - y|}{Range}$$

For categorical attributes:

$$Similarity = \begin{cases} 1, & \text{if attributes match} \\ 0, & \text{otherwise} \end{cases}$$

A weighted similarity score can then be calculated:

$$S = w_1S_1 + w_2S_2 + \dots + w_nS_n$$

where represents the importance of each feature.

Step 3: Retrieve Similar Cases

When a new product is added, the system searches the case database and retrieves products with the highest similarity scores.

For example:

New product → Similar products P1 and P2

Step 4: Reuse Previous Cases

The pricing information and market behaviour of similar products can be reused.

For example:

If similar products had high demand at ₹20,000–₹22,000, the system can use this historical information when determining the price of the new product.

Step 5: Adapt the Solution

The system can modify the previous case according to the new product's characteristics.

This follows the basic CBL process:

Retrieve → Reuse → Revise → Retain

- Retrieve: Find similar products.
- Reuse: Use previous pricing information.
- Revise: Adjust the price according to current market conditions.
- Retain: Store the new product case for future use.

Scaling with Increasing Product Data

As product data increases, searching every case becomes expensive.

The system can improve scalability using:

- Feature indexing
- Clustering
- Vector databases
- Approximate nearest-neighbour search
- Removing duplicate cases
- Efficient case retrieval techniques

Conclusion

Case-Based Learning detects similar products by comparing their attributes with previous cases. Historical product and pricing information can be reused and adapted to new products. Indexing, clustering, and efficient retrieval methods allow the system to handle increasing product data.

5. A logistics company needs to forecast delivery times based on location and traffic patterns. Propose a solution using Radial Basis Function networks or Locally Weighted Regression. Explain how the model captures non-linear relationships and improves prediction reliability under varying conditions.

Solution

A logistics company can use a Radial Basis Function (RBF) network to predict delivery time based on location, distance, traffic, weather, vehicle type, and time of day.

RBF networks are suitable because delivery time often has non-linear relationships with these factors.

Step 1: Collect Input Data

The model can use:

- Pickup location
- Delivery location
- Distance
- Traffic level
- Time of day
- Weather condition
- Vehicle type
- Historical delivery time

Example:

Distance	Traffic	Weather	Time	Delivery Time
----------	---------	---------	------	---------------

5 km	Low	Clear	Morning	20 min
------	-----	-------	---------	--------

10 km	Medium	Clear	Evening	45 min
-------	--------	-------	---------	--------

15 km	High	Rain	Evening	90 min
-------	------	------	---------	--------

Step 2: RBF Network Structure

An RBF network has three main layers:

Input Layer → RBF Hidden Layer → Output Layer

The input layer receives delivery information.

The hidden layer calculates how close the input is to different centre points.

The output layer predicts delivery time.

Step 3: Calculate RBF Activation

A Gaussian radial basis function can be written as:

$$\phi(x) = e^{-\frac{\|x-c\|^2}{2\sigma^2}}$$

where:

- x = input vector
- c = centre of the RBF
- σ = spread parameter

When the input is close to a centre, the RBF produces a high activation.

Step 4: Predict Delivery Time

The final output can be calculated as:

$$y = \sum_{j=1}^m w_j \phi_j(x) + b$$

where:

- w_j = output weight
- $\phi_j(x)$ = RBF activation
- b = bias
- y = predicted delivery time

Capturing Non-Linear Relationships

Delivery time does not increase linearly with distance.

For example:

- 5 km with low traffic → 20 minutes
- 5 km with heavy traffic → 45 minutes
- 15 km with heavy traffic and rain → 90 minutes

The RBF network captures these complex relationships through its hidden-layer functions.

Improving Reliability

The model can be updated using recent delivery data.

Reliability can be improved by:

1. Including real-time traffic information.
2. Updating the model with recent delivery records.
3. Handling missing sensor/GPS data.
4. Removing abnormal delivery records.
5. Retraining when traffic patterns change.
6. Using validation data to evaluate prediction errors.

Example

Suppose a delivery request has:

- Distance = 12 km
- Traffic = High

- Weather = Rain
- Time = 7 PM

The RBF network compares these conditions with historical patterns and produces a predicted delivery time, such as:

Predicted delivery time = 75 minutes

Conclusion

An RBF network can effectively predict delivery time because it captures non-linear relationships between distance, traffic, weather, and other factors. By continuously incorporating recent delivery and traffic data, the system can provide reliable predictions under changing conditions.